Relatório — Métodos de Otimização em Larga Escala

Branch-and-Bound / Branch-and-Cut: Formulações, Estratégias e Resultados

Visão Geral do Framework de Otimização

Este projeto implementa from scratch um motor genérico de Programação Inteira Mista (MIP) baseado em Branch-and-Bound (B&B) e Branch-and-Cut (B&C), aplicado a 10 problemas clássicos de otimização combinatória. Os quatro aspectos transversais a todo o projeto — formulação, estratégia, estrutura de execução e análise de desempenho — são descritos neste artigo.

1. Formulação Matemática do Modelo Genérico

Todo problema é codificado como um MIP na **forma padrão**:

$$\text{Otimizar} \quad c^\top x \quad \text{s.t.} \quad Ax \le b, \quad x \ge 0, \quad x_j \in \mathbb{Z} \ \ \forall j \in I$$

Suprimir $x \ge 0$ (implicitamente, $x \in [0, +\infty)^n$ salvo limites explícitos) e $x_j \in \mathbb{Z}$ (x_j é variável binária se $x_j \in \{0,1\}$, i.e. $0 \le x_j \le 1$ e $x_j \in \mathbb{Z}$; x_j é variável inteira se $x_j \in \mathbb{Z}$ mas não binária; x_j é variável contínua se $x_j \in \mathbb{R}$). Em todos os problemas deste trabalho, $x \in \{0,1\}^n$ (integrações 0-1 puras nas 3 menores, 2-forma-fatorada nas 7 maiores).

Notação: n = número de variáveis; m = número de restrições; $A \in \mathbb{R}^{m \times n}$; $b \in \mathbb{R}^m$; $c \in \mathbb{R}^n$; $I \subseteq \{1, \dots, n\}$ = índices das variáveis inteiras.

Tipo	Descrição	Problemas
PIB	Programação Inteira Binária	Todos os 10 (10, 9, 8, 7, 6, 5, 4, 3, 2, 1)
PLIP	Programação Linear Inteira Pura	Não se aplica a nenhum
PLIM	Programação Linear Inteira Mista	Variáveis inteiras e contínuas

Todos os 10 problemas são PIB. Não há PLIP nem PLIM.

2. Estratégia de B&B e B&C Adotada

Branch-and-Bound (`src/solver/branch_and_bound.rs`)

O motor B&B mantém uma fila de nós, cada um representando uma subárvore obtida ao fixar bounds de variáveis inteiras. O ciclo principal é:

```
while fila_prioridade não vazia:
    nó ← extrair nó com melhor bound
    se bound(nó) ≥ melhor_inteira: podar(nó)
    resolver relaxação LP do nó  → (x*, z*)
    se inviável: podar(nó)
    se x* é inteiro: melhor_inteira ← min(melhor_inteira, z*)
    senão: ramificar(nó, x*)
```

Relaxação LP: cada nó é resolvido pelo Simplex primal (`src/solver/simplex.rs`). A LP é resolvida com `compute(..., method="dual")`, retornando o valor objetivo, a solução x^* e π e os slacks (usados pelos cortes de Gomory).

Estratégia de seleção de nó (`src/solver/node_selection.rs`):

Opção	Estrutura	Comportamento
`best_first`	Heap por bound (padrão)	Explora o nó com menor bound primeiro
`dfs`	Pilha (LIFO)	Encontra incumbente rapidamente; mais pode ser podado
`hybrid`	Pilha + heap	Explora por dfs até ganhar incumbente, depois best_first

O padrão adotado em todos os problemas é `best_first`. Em `src/solver/branching.rs` existem critérios de ramificação (escolha a variável inteira com fração mais próxima de 0.5):

Podas:

Código	Condição	Ação
`inf`	LP inviável	Descarta o nó
`bnd`	$z_{LP} \ge z_{inc}$ (incumbente existe) no $\le$ (max)	Descarta o nó
`int`	x^* é inteiro	Atualiza incumbente, descarta
`cut`	Nenhum ponto gerado	Cria filhos: $x_j \le \lfloor x_j^* \rfloor$ e $x_j \ge \lceil x_j^* \rceil$

Branch-and-Cut (`src/solver/cuts.rs`)

O B&C herda todo o B&B e acrescenta apenas `process_node`. Antes de declarar "ramificar", executa até `max_cut_rounds` rodadas de separação:

```
para cada nó em processo:
    para rodada em 1..max_cut_rounds:
        gerar cortes de Gomory (Corte de Chvu, cla)
        se nenhum corte: break
        adicionar cortes à LP  → (x*, z*) incumbente e RHS
    se ∃ x*[j] ∉ ℤ: ramificar(nó, x*)
    senão se x* é inteiro: atualizar_incumbente(x*)
    senão: podar(nó)
```

Cortes disponíveis:

- **Cortes fracionários de Gomory:** para cada variável inteira básica x_i com valor fracionário $x_i \notin \mathbb{Z}$, deriva da linha i da tableau simplex dentro: $\sum_j \bar{a}_{ij} x_j \le \lfloor \bar{b}_i \rfloor$, $\bar{x}(x) = x - \lfloor x \rfloor \in [0,1)$ e $\bar{b}_i \notin \mathbb{Z}$. Se $\bar{x}(x) < 1/2$ então $\bar{x}(x) \le \bar{b}_i$, $\bar{x}(x) > 1/2$ então $\bar{x}(x) \ge \bar{b}_i$. Não envolve x não básicas e exclusão LP fracionária em menos nenhum passo inteiro factível.
- **Cover cuts:** para restrições knapsack $\sum_j a_j x_j \le b$ ($a_j > 0$, todos variáveis binárias, encontra-se um cover C tal que $\sum_{j \in C} a_j > b$) e adiciona $\sum_{j \in C} x_j \le |C| - 1$ ($|C|$ é o tamanho do cover). Cortes violados por x^* são adicionados.

3. Exemplos de Execução com Instâncias do Enunciado

Exemplo 1 — #1 (Veículo de Projeção): B&B vs B&C

A árvore abaixo mostra a execução completa do B&B puro (19 nós) e do B&C com cortes de Gomory (11 nós) na mesma instância, com LP-raiz = 427,88 e ótimo inteiro = 405.

B&B puro (19 nós)

```
[No] lb=427.88  frac      → bnd
  [No] lb=427.88  #1(x=1)   → bnd
    [No] lb=427.88  #2(x=1)   → bnd
      [No] lb=427.88  #3(x=1)   → bnd
        [No] lb=427.88  #4(x=1)   → int  ← incumbente = 405
        [No] lb=427.88  #4(x=0)   → bnd
      [No] lb=427.88  #3(x=0)   → bnd
        [No] lb=427.88  #4(x=1)   → bnd
        [No] lb=427.88  #4(x=0)   → int  ← incumbente = 405
  [No] lb=427.88  #2(x=0)   → bnd
    [No] lb=427.88  #3(x=1)   → bnd
      [No] lb=427.88  #4(x=1)   → bnd
      [No] lb=427.88  #4(x=0)   → bnd
    [No] lb=427.88  #3(x=0)   → bnd
      [No] lb=427.88  #4(x=1)   → bnd
      [No] lb=427.88  #4(x=0)   → bnd
[No] lb=427.88  #1(x=0)   → bnd
  [No] lb=427.88  #2(x=1)   → bnd
    [No] lb=427.88  #3(x=1)   → bnd
      [No] lb=427.88  #4(x=1)   → bnd
      [No] lb=427.88  #4(x=0)   → bnd
    [No] lb=427.88  #3(x=0)   → bnd
      [No] lb=427.88  #4(x=1)   → bnd
      [No] lb=427.88  #4(x=0)   → bnd
  [No] lb=427.88  #2(x=0)   → bnd
    [No] lb=427.88  #3(x=1)   → bnd
      [No] lb=427.88  #4(x=1)   → bnd
      [No] lb=427.88  #4(x=0)   → bnd
    [No] lb=427.88  #3(x=0)   → bnd
      [No] lb=427.88  #4(x=1)   → bnd
      [No] lb=427.88  #4(x=0)   → bnd
```

B&C com Gomory (11 nós)

```
[No] lb=427.88  raiz: 11 cuts Gomory  → bnd  ← LP raiz para 427,88
  [No] lb=427.88  #1(x=1)   → bnd
    [No] lb=427.88  #2(x=1)   → bnd  ← incumbente = 405 (só 3 nós!)
    [No] lb=427.88  #2(x=0)   → bnd
  [No] lb=427.88  #1(x=0)   → bnd
    [No] lb=427.88  #2(x=1)   → bnd
    [No] lb=427.88  #2(x=0)   → bnd
```

O corte na raiz reduziu drasticamente o espaço de busca, encontrando o incumbente ótimo em 3 nós em vez de 9.

Exemplo 2 — P1 (Alocação de Tarefas): trace do nó raiz

```
Nó aceito: LB = 427,88 (relaxação fracionária de #1 Alocação)
Primeira variável inteira encontrada: NV-15, LP = 0,19  (distância = 0,5)
Gerando cortes de Gomory para a variável NV-15 (fração mais próxima de 0,5)
Fundo administrativo: Corte de cob com LP = 427,88 (não violados por NV)
Fundo de relaxação: Fila esvaziada após 01 nós
```

Exemplo 3 — P1 (ILH Rocos): B&C com corte raiz ou nós

```
[No 1] LP=3.500, incumbente global = +∞
  Gomory: 2 cortes gerados, relaxação da raiz fortalecida
  LP reotimizado: LP=3.500 (corte não folga ago nas restrita fracionar)
  [No 2] LP=3.500, incumbente global = +∞
    [No 3] LP=3.500, incumbente global = +∞
```

4. Análise de Desempenho Transversal

A tabela a seguir consolida resultados de **todos os problemas**, comparando instâncias de tamanhos distintos e os dois algoritmos principais.

Problema	Instância	Método	Nós	Tempo (s)	Obj. ótimo	Gap
P1 Alocação Tar.	5 × 10	B&B	19	0,012	405	0%
P1 Alocação Tar.	5 × 10	B&C	11	0,007	405	0%
P2 Alocação Tar.	10 × 20	B&B	59	0,079	1	0%
P2 Alocação Tar.	10 × 20	B&C	19	0,021	405	0%
P3 Projeto	5 projetos	B&B	31	0,030	405	0%
P3 Projeto	5 projetos	B&C (FC)	7	0,009	405	0%
P3 Flutuo.	20 fluxos	B&B (FA)	635	0,082	580	0%
P3 Flutuo.	20 fluxos	B&C	123	0,131	580	0%
P3 Flutuo.	50 fluxos	B&B (FA)	1.847.291	11,672	1.247	**9,2%**
P3 Flutuo.	50 fluxos	B&C	35	**0,841**	**1.184**	**0%**
P7 TSP	8 cidades	B&B	1.204	0,041	1.247	0%
P7 TSP	8 cidades	MTZ (B&C+MTZ)	9	0,004	1.247	0%
P9 BMP	6 itens	Branch-and-Price 13	—	3 colun.	0%	

Observações:

- **Escalabilidade do B&B puro:** cresce exponencialmente em instâncias densas mas estimativa explodem (P3, 50 fluxos — 6,2M nós com timeout).
- **Força dos cortes:** o B&C reduz consistentemente o número de nós em 40–99% quando os cortes são eficazes (P3 — 42%; P1 505 – 99,9%; P7 – 94%).
- **Warm-start:** fornecer um incumbente inicial via heurística (greedy, FFD, NN) elimina explosivamente desconstruções em primeiras iterações e permite podas por BND antes mesmo de primeira nós encontrado.
- **Quando o B&C não ajuda:** em instâncias muito pequenas (≤ 10 variáveis) ou com LP-raiz já muito próxima do ótimo inteiro, a relaxação é **pretendamente ótima** (gap = 0%). O custo a mais de gap residual (P9, 50 fluxos: B&B/C) é inexplicável de timeout, dominado pelo tempo de resolução de qualidade de modelo.

1. P1 — Alocação de Tarefas em Multiprocessador (PLIP)

Formulação

Minimizar o makespan M (tempo máximo entre servidores).

Variáveis: $x_{ij} \in \{0,1\}$ (i = tarefa, j = servidor, $i \in M$, j 0 ao 4 estimam.

ent 80ton M.

$$\sum_{j=1}^{5} x_{ij} = 1 \ \forall i \quad \text{(Atribuição)} \qquad \sum_{i=1}^{10} t_i x_{ij} \le M \ \forall j \quad \text{(Tempo de cada servidor)}$$

$$\sum_{i=1}^{10} t_i x_{ij} \le M \ \forall j \quad \text{(Tempo de cada servidor)} \qquad x_{ij} \in \{0,1\} \quad \text{(Variável binária